Agent teams in Claude and broader multi-agent systems

Executive summary

The most important conclusion is rather unglamorous, which usually means it is probably true: **most teams should start with one good agent, good tools, and disciplined context management, then add more agents only when there is a clear need for separation of context, parallel work, specialised roles, or independent critique**. Anthropic's own guidance argues for "simple, composable patterns" over elaborate agent frameworks, and warns that agents bring higher cost and compounding error risk; Microsoft makes a similar point from a workflow angle, advising that if a plain function can do the job, it should. [1]

On the Anthropic side, there is now a **real spectrum of "native" support**, not a simple yes-or-no. At one end, the **Messages API** gives direct model access and leaves orchestration to you. In the middle, the **Claude Agent SDK** gives you Claude Code's agent loop, tools, context management, hooks, subagents, observability, and cost tracking, but it still runs in infrastructure you operate. At the more managed end, **Claude Managed Agents** gives you Anthropic-hosted runtime, containers, stateful sessions, prompt caching, compaction, memory stores, and built-in tools. On top of that, Anthropic now exposes **two distinct multi-agent surfaces**: **Claude Code agent teams**, which are **experimental and disabled by default**, and **Managed Agents multiagent sessions**, which are a **Research Preview** with access gating and only **one level of delegation** today. [2]

That matters because "native Anthropic agent teams" are not yet one settled, fully general product concept. **Claude Code agent teams** are oriented around human-supervised coding sessions, with a team lead, teammates, a shared task list, a mailbox, direct teammate messaging, and hooks for task quality gates. **Managed Agents multiagent**, by contrast, is an API feature where a coordinator spawns isolated worker threads inside a shared container; each worker keeps its own conversation history and configuration, and tools/context are not shared between agents. Those are related ideas, but not the same abstraction. [3]

Across providers and frameworks, the most useful mental model is not "what framework do I use?" but **"what coordination pattern does the work require?"**. The four patterns that best organise the landscape are: **central orchestrator or planner-executor**, **sequential pipeline**, **parallel specialists or debate**, and **swarm or decentralised shared-context systems**. This four-part taxonomy is an analytic synthesis rather than an official Anthropic taxonomy, but it maps cleanly onto Anthropic's workflow patterns, OpenAI's handoffs versus agents-as-tools, Microsoft's orchestration catalogue, and Google ADK's multi-agent primitives. [4]

On communication, the practical answer is also more layered than people often assume. Agent teams communicate through a mix of **natural-language messages**, **structured tool or handoff calls**, and **shared state or artefacts**. **MCP** standardises **agent-to-tool** communication; **A2A** standardises **agent-to-agent** communication. Anthropic itself frames MCP as an open protocol for connecting assistants to tools and data; Google frames A2A as its complement for agent interoperability, with concepts such as **Agent Cards**, **tasks**, **artifacts**, and streaming updates. In other words, if someone says "we need agent protocols", the sensible reply is "for which layer?". [5]

The hidden cost of agent teams is that they are not only a prompting problem. They are also a **distributed systems problem**: ownership, message routing, retries, shared state, observability, evaluation, failure containment, and security all become materially harder. Anthropic's engineering posts are especially clear on this. Their multi-agent research system needed explicit delegation rules, effort budgets, better tool descriptions, tracing, memory, and eventually filesystem-based artefacts just to avoid duplicated work, bottlenecks, and the "game of telephone". Claude Code's own docs say agent teams consume **significantly more tokens** and add coordination overhead. That is the anti-hype sentence worth remembering. [6]

Source map

The research is anchored primarily in official documentation, engineering posts, and protocol specifications. I have used Tier 2 sparingly and only for supplementary colour rather than load-bearing claims.

Tier 1 sources

Source	Why it matters	Main sections supported
Anthropic, **Building effective agents** [7]	Anthropic's clearest primary statement on when to use workflows versus agents, and on prompt chaining, routing, parallelisation, orchestrator-workers, and evaluator-optimizer.	Executive summary; paradigms; failure modes
Claude Code Docs, **Orchestrate teams of Claude Code sessions** [8]	Primary source for Anthropic's experimental "agent teams" feature: status, architecture, shared task list, mailbox, direct teammate messaging, and limitations.	Anthropic-specific findings; communication; failure modes
Claude Code Docs, **Create custom subagents** and **Subagents in the SDK** [9]	Defines what subagents are, why they exist, and how they differ from peer-to-peer teams.	Anthropic-specific findings; paradigms

Source	Why it matters	Main sections supported
Claude Code Docs, **How Claude remembers your project**, **Extend Claude with skills**, and **Hooks reference** [10]	Canonical source for memory, skills, hooks, JSON hook payloads, subagent context injection, and task gating.	Anthropic-specific findings; communication; failure modes
Anthropic Engineering, **How we built our multi-agent research system** [11]	Best primary source on Anthropic's own production multi-agent design, delegation, tracing, scaling rules, bottlenecks, and artefact-based communication.	Anthropic-specific findings; paradigms; communication; failure modes
Claude API Docs, **Claude Managed Agents overview**, **multiagent sessions**, **memory**, **tools**, and **migration** [12]	Primary source for Anthropic's most managed agent runtime, including what Anthropic hosts versus what the developer still owns, plus the current preview multi-agent API.	Anthropic-specific findings; native vs DIY; communication
Anthropic Research, **Trustworthy agents in practice** and Anthropic Engineering, **Effective harnesses for long-running agents** [13]	Strong primary sources on oversight, permissions, ambiguity, long-running state, evaluation, and agent reliability.	Failure modes; native vs DIY; communication
OpenAI, **Agents SDK**, **Orchestration and handoffs**, and **Function calling** [14]	Useful counterpoint to Anthropic: clean distinction between handoffs and agents-as-tools, and clear description of JSON-schema-based tool/function calling.	Paradigms; communication; native vs DIY
Model Context Protocol specification and Anthropic's MCP launch post [15]	Canonical source for MCP as agent-to-tool protocol, including JSON-RPC 2.0 and host/client/server concepts.	Communication
Google, **A2A** blog and official A2A docs/specification [16]	Primary source for agent-to-agent interoperability: Agent Cards, tasks, artifacts, JSON-RPC versus HTTP/REST bindings, and SSE streaming.	Communication; paradigms
Google ADK, **Multi-agent systems**, **Events**, and **A2A**	Strong official reference for coordinator, pipeline, parallel	Paradigms; communication

Source	Why it matters	Main sections supported
docs [17]	fan-out, generator-critic, shared session state, agent transfer, and agent-as-tool patterns.	
Microsoft Agent Framework and AutoGen docs [18]	Clear official taxonomy of orchestration patterns, plus high-quality explanations of group chat, reflection, and handoff orchestration.	Paradigms; communication; failure modes

Tier 2 supplementary sources

Source	Why it matters	Main sections supported
METR, **Red-Teaming Anthropic's Internal Agent Monitoring Systems** [19]	Independent, reputable supplementary evidence that observability and monitoring are not merely theoretical nice-to-haves in agent systems. I treat it as supporting colour, not as a basis for architectural claims.	Failure modes

Anthropic-specific findings

The simplest accurate picture of Anthropic's stack

Anthropic now offers at least **three practical levels of agent building**, plus **two preview/experimental multi-agent surfaces**. If you reduce the landscape to "Claude supports teams" or "Claude does not support teams", you lose the most important distinctions. The present reality is a ladder: **Messages API** for custom loops; **Claude Agent SDK** for Anthropic's loop running in your infrastructure; **Claude Managed Agents** for Anthropic-hosted runtime; then **Managed Agents multiagent** and **Claude Code agent teams** as newer, narrower multi-agent layers on top. [20]

Anthropic surface	Who owns the runtime and orchestration	What Anthropic handles	Multi-agent status	Best mental model
Messages API [21]	You	Direct prompting only; tool use exists, but you implement the loop/tool execution yourself.	None built in.	Raw materials for DIY orchestration.
Claude Agent SDK	Mostly you	Claude Code's tools, agent loop, context	**Subagents** supported;	Hybrid: Anthropic loop,

Anthropic surface	Who owns the runtime and orchestration	What Anthropic handles	Multi-agent status	Best mental model
[22]		management, hooks, subagents, sessions, observability, and cost tracking.	richer multi-agent patterns still largely your design.	your system.
Claude Managed Agents [23]	Anthropic	Managed infrastructure, containers, prompt caching, compaction, stateful sessions, event history, memory mounts, built-in tools.	Single-agent is mature; multi-agent is separate preview.	Hosted harness.
Managed Agents multiagent sessions [24]	Anthropic	Coordinator/worker threading, event streams, routing across agent threads.	**Research Preview**, access-gated, **one level of delegation only**.	Native but still constrained multi-agent.
Claude Code subagents [25]	Claude Code / you as operator	Isolated worker contexts, summaries back to caller, task-specific instructions.	Official feature, but not peer-to-peer teams.	Bounded helpers.
Claude Code agent teams [26]	Claude Code / you as operator	Team lead, teammates, shared task list, mailbox, direct teammate interaction, hook integration.	**Experimental**, disabled by default.	Human-supervised team coordination in the coding product.

This means that **Anthropic absolutely does provide agent-team functionality today**, but it does so in **different ways with different maturity levels**. If you are writing a technical blog post, that subtlety matters. “Claude has agent teams” is true, but incomplete; “Claude has production-grade, general-purpose native multi-agent orchestration across all surfaces” would overstate the current state of the platform. [27]

What Anthropic officially supports today

The most mature, clearly supported Anthropic primitives for agentic systems are not “teams” but **single-agent loops with structured extension points**: **always-on project context via CLAUDE.md**, **auto memory**, **on-demand skills**, **MCP integrations**, **subagents**, and **hooks**. Anthropic’s Claude Code docs explicitly describe these as a

layered extension system: CLAUDE.md for persistent context, skills for reusable knowledge and workflows, MCP for external services, and subagents for isolated loops that return summaries. [28]

That is an important design clue. Anthropic is effectively encouraging a mental model in which **specialisation is often achieved without full multi-agent peer networks**. A skill can hold a reusable playbook and only load when needed; a skill can even run in a forked subagent context with `context: fork`, so you get isolation and specialisation without keeping several long-lived agents chatting to one another like over-caffeinated consultants in a glass meeting room. [29]

Hooks show the same philosophy. They are not merely shell callbacks. Claude Code hooks can be shell commands, HTTP endpoints, MCP tool hooks, prompt-based hooks, or experimental agent-based hooks; Claude passes **JSON context** to the hook, and the hook can approve, block, or annotate behaviour. There are even hook events for **SubagentStart/SubagentStop**, **TaskCreated/TaskCompleted**, and **TeammateIdle**, which means Anthropic sees lifecycle governance, validation, and observability as first-class concerns in agentic systems. [30]

How Claude Code teams relate to memory, skills, hooks, and subagents

In Claude Code, **subagents** and **agent teams** solve related but different problems. Subagents run inside a single main session and report results back to the caller. Agent teams are separate Claude Code sessions with their own contexts, plus a **shared task list** and **mailbox** so teammates can message one another directly and self-coordinate. Anthropic's docs are explicit that subagents are better when **only the result matters**, while agent teams are better when workers **need to share findings, challenge one another, and coordinate on their own**. [31]

Memory also behaves differently at different layers. In Claude Code, every session starts fresh, but knowledge can persist through **CLAUDE.md** and **auto memory**; Anthropic notes that **subagents can also maintain their own auto memory**. This is useful for specialisation, but it also means the moment you split work across agents, you are now making design choices about **which knowledge should stay local, which should be shared, and which should be summarised back**. That is not a trivial prompt tweak; it is the beginning of architecture. [32]

What Managed Agents changes

Claude Managed Agents changes the bargain in a more substantial way. Anthropic says Managed Agents provides the **harness and infrastructure** for autonomous work, so instead of building your own loop, tool runtime, and sandbox, you get a managed environment where Claude can read files, run commands, browse the web, and execute code securely. Anthropic also positions Managed Agents as best for **long-running execution**, **cloud infrastructure**, **minimal infrastructure burden**, and **stateful sessions**. [33]

However, the migration docs are valuable because they show precisely **what does not disappear**. Even on Managed Agents, you still control the **system prompt and model**, still define **custom tools** with **JSON Schema**, still inject context via system prompt, files, or skills, and still have to respond to custom-tool events when your application is the executor. In other words, Anthropic can host the harness, but it cannot magically remove the need for application-level design. [34]

What is still experimental or preview-only

Two items deserve especially careful wording in any blog post.

First, **Claude Code agent teams are experimental and disabled by default**, with known limitations around session resumption, task coordination, and shutdown behaviour. Anthropic's docs even note cases where resumed sessions may refer to teammates that no longer exist, or where task state lags and requires manual nudging. [35]

Second, **Managed Agents multiagent sessions are a Research Preview**. They require additional beta headers, access is gated, and the current design supports only **one coordinator level**: a coordinator can call other agents, but called agents cannot themselves call further agents. That is native multi-agent orchestration, but it is not yet a general-purpose recursive agent society. [36]

The safest synthesis is this: **Anthropic officially supports strong building blocks for agentic systems, plus two real but still evolving multi-agent products or APIs. For unusual topologies, cross-provider portability, richer policies, or domain-specific orchestration, custom design is still very much your job.** [37]

General agent team paradigms

The four-pattern taxonomy below is a **synthesis**, not an official Anthropic naming scheme. Anthropic talks in terms of workflows such as prompt chaining, routing, parallelisation, orchestrator-workers, and evaluator-optimizer; Microsoft exposes sequential, concurrent, handoff, group chat, and magentic orchestration; OpenAI distinguishes handoffs from agents-as-tools; Google ADK talks about coordinator, sequential, fan-out/gather, hierarchical decomposition, and generator-critic. Condensing that landscape into four higher-level paradigms produces a useful mental model without pretending the ecosystem has one universal glossary. [38]

Comparison across the four paradigms

Paradigm	What it is	Use it when	Avoid it when	Main strengths	Main failure modes	Anthropic mapping
Central orchestrator / planner-	One lead agent decomposes work,	Task decomposition is dynamic;	The workflow is fixed and simple	Central control, easier policy	Manager bottlenecks, vague delegatio	Claude Code lead + teammate

Paradigm	What it is	Use it when	Avoid it when	Main strengths	Main failure modes	Anthropic mapping
executor	delegates, then synthesises. [39]	specialist boundaries are clear; oversight matters. [40]	enough to script. [41]	enforcement, specialist reuse.	n, duplicated work, aggregation errors. [42]	s; Anthropic Research lead + subagents; Managed Agents coordinator. [43]
Sequential pipeline	Agents or stages run in a defined order; later stages consume earlier outputs. [44]	Work has natural stages such as research → synthesis → critique.	You need dynamic routing or extensive back-and-forth.	Easier debugging, clearer contracts, better predictability.	Latency stacking, brittle stage interfaces, error propagation. [45]	Prompt chaining, evaluator-optimizer, skills in forked subagents, long-running harness stages. [46]
Parallel specialists / debate / red team	Multiple agents examine the same task or related subtasks at once, then vote, critique, or aggregate. [47]	You want breadth, robustness, independent hypotheses, or adversarial checking.	The subproblems are tightly coupled or share too much mutable state.	Better coverage, faster wall-clock time, diverse views.	Token blow-up, duplicated work, weak aggregation, false confidence from consensus. [48]	Agent teams for competing hypotheses; Anthropic Research parallel subagents; Parallelisation “sectioning” and “voting”. [49]
Swarm / decentralised /	Coordination emerges	Exploration is open-ended;	You need strict accountabi	Flexible collaboratio n,	Hardest to reason about,	Claude Code teams are

Paradigm	What it is	Use it when	Avoid it when	Main strengths	Main failure modes	Anthropic mapping
shared-context systems	through shared state, shared threads, messages, or environment changes rather than one strict leader.	remote agents must interoperate; shared environment matters. [50]	lity, deterministic routing, or easy debugging.	interoperability, less central bottleneck.	shared-state pollution, ownership ambiguity. [51]	a controlled local version; Managed Agents threads plus inter-thread messages are a constrained platform version. Full swarm remains mostly custom. [52]

Central orchestrator or planner-executor

This is the most useful "default" multi-agent pattern. Anthropic's **orchestrator-workers** workflow says it plainly: a central LLM dynamically breaks down tasks, delegates them to worker LLMs, and synthesises the results. Anthropic's own Research feature uses exactly this pattern: a lead agent analyses the query, spawns specialised subagents in parallel, and later synthesises findings. OpenAI's "agents as tools" is closely related, except the manager stays owner of the final reply. [53]

Use it when the subtask structure is **not known in advance**, but the system still needs clear top-level ownership. Anthropic highlights coding changes across multiple files and search tasks over many sources as good fits; Managed Agents preview recommends code review, test generation, and research as well-scoped delegated tasks. [40]

Do not use it if the workflow is already fixed enough to script as a pipeline. Microsoft's overview is refreshingly blunt: workflows are for well-defined execution order; if you can write a function to do the job, do that instead of calling in a committee of probability distributions. [41]

The main strength is **control**. The main weakness is **orchestrator failure**. Anthropic's research system found that a lead agent with vague delegation prompts caused

duplication and gaps, and that synchronous execution produced bottlenecks because the lead could not steer workers mid-flight. [42]

Sequential pipeline

Anthropic's **prompt chaining** and **evaluator-optimizer** are canonical examples of this pattern. Work is decomposed into ordered stages: one stage produces a structured intermediate output; the next stage consumes it; optional gates or checks sit between stages. Google ADK's **SequentialAgent** and Microsoft's sequential orchestration formalise the same idea. [54]

This pattern is best when the work already has a natural order: research then synthesis, drafting then critique, extraction then validation, planning then execution. It is usually the easiest pattern to explain to a technical audience because it looks most like ordinary software pipelines. [45]

Its main strength is that **contracts are easier to make explicit**. Its main weakness is that **mistakes cascade**. If the first stage extracts the wrong thing, the later stages can be beautifully wrong at great expense and with remarkable confidence, which is not always the sort of confidence one hopes to encourage. Anthropic's long-running harness work ends up solving this by storing progress structurally, working incrementally, and validating end-to-end before marking things done. [55]

Parallel specialists, debate, and red team

Anthropic's **parallelisation** pattern has two variants: **sectioning**, where independent subtasks run in parallel, and **voting**, where the same task is run multiple times for higher confidence or diversity. Microsoft's concurrent orchestration says much the same, positioning parallel agents for diverse perspectives, ensemble reasoning, or voting. OpenAI's debate work provides a conceptual academic root for adversarial multi-agent checking. [47]

Use this pattern when you care about **robustness more than tidiness**: competing bug hypotheses, alternative research directions, policy critique, security review, or devil's-advocate analysis. Anthropic's own Claude Code team examples explicitly mention research and review, debugging with competing hypotheses, and cross-layer coordination. [56]

Do not use it for tightly coupled tasks where workers continually need to edit the same artifact or depend on one another's mutable state. Anthropic's docs explicitly say agent teams are worse for sequential tasks, same-file edits, or work with many dependencies, where a single session or subagents are more effective. [57]

The strength is breadth and wall-clock speed. The cost is obvious: more agents means **significantly more tokens**, more aggregation work, and more chances for "independent" workers to rediscover the same thing in slightly different words. Anthropic's own research

system had to add effort budgets and clearer delegation to stop workers from fanning out absurdly. [58]

Swarm, decentralised, and shared-context systems

This is the least tidy pattern and therefore, for certain problems, the most realistic one. Microsoft AutoGen's **group chat** is essentially a shared message thread where specialised agents publish to the same topic and a manager selects the next speaker. Google A2A introduces remote-agent interoperability via **Agent Cards**, **tasks**, **artifacts**, and messages, while Google ADK exposes **shared session state**, **agent transfer**, and **events** as communication primitives. [59]

Use this pattern when the system must support looser collaboration, environment-mediated coordination, or remote interoperability. It is especially relevant when agents are not all part of one colocated runtime. A2A's whole point is that agents built by different vendors and frameworks should still be able to discover one another and collaborate. [60]

Do not use it if you need easy debugging, tight compliance boundaries, or simple accountability. Shared-state and pub-sub systems are powerful, but they are where "agents" starts to become "distributed systems", and that is where many otherwise sensible prototypes come to a rather abrupt philosophical stop. Google ADK itself warns about using shared state keys carefully to avoid race conditions, and Microsoft's group chat documentation makes the managerial turn-selection logic explicit because otherwise the pattern becomes opaque quickly. [61]

Native vs DIY orchestration

The real comparison is three-way, not two-way

In practice, the useful comparison is **native managed**, **DIY**, and **hybrid**.

Native managed means Anthropic owns the runtime or coordination surface: **Claude Managed Agents** and, in narrower ways, **Managed Agents multiagent** or **Claude Code agent teams**. **DIY** means you own the orchestration logic and usually the runtime too: the **Messages API** is the purest version of that. **Hybrid** means Anthropic provides the loop and tools, but you still architect the system: the **Claude Agent SDK** is the clearest example. [62]

Trade-off analysis

The table below is partly documentary and partly interpretive. Where a row reflects direct product capability, it is sourced directly; where it reflects engineering judgement about the trade-off, that is an inference from the documented capabilities and constraints.

Dimension	Native managed Anthropic	DIY orchestration	Hybrid
Control	Lowest to medium. Anthropic hosts the	Highest. You own topology, memory,	High. Anthropic owns the loop mechanics;

Dimension	Native managed Anthropic	DIY orchestration	Hybrid
	runtime/harness and imposes platform constraints such as current multiagent preview limits. [63]	retries, approvals, and failure handling. [21]	you own higher-level design. [64]
Reliability	Better out of the box for long-running sessions and stateful agent runtime because Anthropic provides compaction, prompt caching, containers, and persistent event history. [33]	Reliability is whatever you can engineer. This can exceed native on narrow domains, but only with work.	Often the best practical middle ground for production teams that want Anthropic's loop but custom policies. [65]
Observability	Good, but not complete by default. Managed Agents exposes session streams and thread streams; Claude Code and the Agent SDK expose OTel and cost tracking. [66]	Potentially best, because you can instrument everything, but you must actually do it.	Strong, because Anthropic gives instrumentation hooks while you still control your app-level traces. [67]
Flexibility	Lower. Managed Agents multiagent is preview-only and currently one-level; Claude Code teams are product-specific and experimental. [68]	Highest.	High, but bounded by Anthropic's loop/tool model.
Cost	Lower engineering cost; runtime/token cost is **not automatically lower**. Anthropic explicitly says agents and teams can cost more, and Claude Code teams use significantly more tokens. [69]	Higher engineering cost, but potentially lower runtime waste if your orchestration is tighter.	Moderate on both axes.
Latency	Better operationally because you skip platform plumbing; not a cure for reasoning latency or coordination delay. Preview multiagent and teams can still bottleneck. [70]	You can optimise aggressively, but only by building it.	Often best if you need some platform help plus custom parallelism.

Dimension	Native managed Anthropic	DIY orchestration	Hybrid
Debuggability	Better than ad hoc prototypes, but still limited when behaviour is emergent. Anthropic's own teams rely heavily on tracing and evals. [71]	Potentially best, but only if you invest in traces, stored artefacts, and evals.	Usually strong, because you can add your own diagnostics around Anthropic primitives.
Maintenance	Lower infrastructure burden; versioned agents and managed resources help. [72]	Highest maintenance burden.	Moderate.
Portability	Lowest. Managed Agents is available only through the direct Claude API. [73]	Highest across providers and models.	Better than managed. Agent SDK can authenticate through Bedrock, Vertex AI, and Azure Foundry. [74]

Opinionated guidance

Choice	It is the right choice when…	It is the wrong choice when…
Use Anthropic-native managed functionality	You want a hosted runtime, long-running sessions, stateful containers, less platform engineering, and are comfortable with Anthropic-specific constraints. [33]	You need broad provider portability, unusual topologies, or mature fully general multi-agent support today. [75]
Use DIY orchestration	Your business logic is unusual, compliance is strict, routing is domain-specific, you need provider portability, or you want total control of memory and handoff semantics.	You mostly need a straight agent loop with tools and are about to spend weeks rebuilding what Anthropic already ships. [76]
Use hybrid	You want Anthropic's loop, tools, hooks, subagents, and observability, but still want to own system architecture and keep room for migration. [77]	You either want purely hosted simplicity or purely bespoke control.

My practical recommendation is therefore simple: **use native managed features when infrastructure is the pain; use DIY when topology and policy are the pain; use hybrid when you want Anthropic's loop but do not want to marry your entire architecture to Anthropic's opinion of how the future should look.** [78]

Agent communication model

How agents communicate

At the highest level, agent communication happens in **three broad forms**: **plain language messages**, **structured invocation formats**, and **shared state or artefacts**.

Plain-language messaging is the most obvious. Claude Code agent teams let teammates **message each other directly** through a mailbox and shared task system; Managed Agents multiagent exposes **thread message sent/received** events; A2A includes agent-to-agent messages with content "parts". This style is flexible and expressive, which is why it is usually used for delegation instructions, follow-up questions, and explanation. Its weakness is that it is less precise and therefore more failure-prone. [79]

Structured invocation is the second layer. Anthropic tool definitions use **JSON Schema** for tool inputs, and Anthropic's **strict tool use** guarantees schema-conformant tool arguments. OpenAI's function calling likewise uses tools defined by a **JSON schema**. In Microsoft's handoff orchestration and OpenAI's handoffs, control transfer is also represented through function-style tool calls rather than vague prose alone. In practice, high-reliability systems often make the routing act structured even when the surrounding content remains natural language. [80]

Shared state and artefacts are the third layer, and they are often the most underrated. Anthropic's research system explicitly recommends having subagents write outputs to a filesystem or external system and return **lightweight references** rather than piping everything through the lead agent's conversational buffer. Managed Agents memory stores are mounted as directories, versioned, auditable, and editable through the API. Google ADK exposes **shared session state** as a primary communication mechanism, while A2A treats task outputs as **artifacts**. This is how mature systems avoid the "game of telephone". [81]

When agents communicate

The timing patterns matter as much as the format.

Synchronous calls are the simplest. A manager calls a specialist as a tool, waits, then continues. OpenAI's "agents as tools" and Claude subagents are both examples in spirit. This is easy to reason about, but it creates bottlenecks. Anthropic's own research team notes that synchronous lead-agent execution limited mid-flight steering and blocked the system on slow workers. [82]

Sequential handoffs transfer ownership from one agent to another. OpenAI formalises the distinction: use **handoffs** when the specialist should own the next response, and **agents as tools** when the manager should stay in control. Microsoft's handoff orchestration makes the same distinction and preserves the conversation across agent switches. This is conceptually elegant for triage, customer support, and role transitions. [83]

Event-driven or streaming coordination is the native mode for looser systems. A2A supports long-running tasks, state updates, and streaming via SSE; Managed Agents multiagent surfaces session and thread events; Claude Code teams automatically deliver teammate messages and idle notifications; Google ADK uses immutable **events** as the unit of information flow. This mode is better for long-running or asynchronous work, but harder to debug and test. [84]

Shared-state polling and checkpoints are the least glamorous but often the most robust. ADK pipelines and loops use shared session state; Anthropic's long-running harness work uses JSON feature lists, progress files, git commits, memory, and end-to-end checks; Anthropic's Plan Mode moves approval from individual actions to the overall plan. Human-in-the-loop checkpoints are not a luxury add-on here; they are often the difference between a useful agent and a highly energetic misunderstanding. [85]

What agents pass to each other

What should pass between agents depends on the coordination pattern.

Passing the **full conversation history** maximises continuity. Microsoft's handoff model preserves the conversation across specialists, and Managed Agents threads persist their own histories over time. The trade-off is obvious: more context helps continuity, but it also grows cost, latency, and the chance that irrelevant prior material contaminates the next step. [86]

Passing a **compressed summary** is often better. Claude Code subagents work precisely because they do their heavy reading in isolated context and return only a summary to the main conversation. Anthropic's context-engineering post also frames context as an iterative curation problem, not a "throw in everything and hope" problem. Anthropic's long-horizon systems summarise finished phases and store essentials externally before proceeding. [87]

Passing **task-specific context and intermediate artefacts** is usually the practical sweet spot. Feature lists in JSON, progress logs, stored reports, code patches, retrieved files, or A2A artifacts all serve as stable handoff objects. This improves fidelity and auditability because the receiving agent can inspect the artifact directly rather than trusting a summary alone. Anthropic's long-running harness and research-system write-ups both push strongly in this direction. [88]

Passing **tool outputs, decisions, and error states** is what makes agent systems operational rather than theatrical. Managed Agents' thread-aware tool confirmation flow requires you to echo `session_thread_id` to the waiting thread; Claude hooks receive JSON event payloads and can block completion; A2A treats task status as a first-class lifecycle. By contrast, **confidence scores** are a common design choice but not a standard requirement in the primary protocols reviewed here. That is worth spelling out carefully: confidence can be useful, but it is a policy you design, not something MCP or A2A hands you by default. [89]

Protocols and where they fit

The most useful conceptual split is this:

- **MCP** is primarily **agent-to-tool / agent-to-context** infrastructure. Anthropic's official spec defines hosts, clients, and servers and uses **JSON-RPC 2.0**; Google explicitly describes A2A as complementary to MCP rather than a replacement; OpenAI now supports remote MCP servers and connectors as tool surfaces. [90]
- **A2A** is primarily **agent-to-agent** infrastructure. It defines **Agent Cards** for discovery, **tasks** with lifecycle, **artifacts** as outputs, and standard bindings including **JSON-RPC**, **gRPC**, and **HTTP+JSON/REST**, plus streaming via **SSE**. [91]

That division is cleaner than much of the current online chatter, which often treats every protocol as if it were solving every coordination problem simultaneously. It is not. One protocol helps an agent talk to tools; the other helps agents talk to agents. That is already enough complexity for one blog post without trying to persuade the reader that everything is secretly one thing. [92]

Failure modes

Why agent teams break in practice

The common failure modes are not mysterious. Anthropic's engineering notes read less like science fiction and more like the diary of a team that discovered, one bug at a time, that extra agents multiply management problems.

The first is **coordination drift**. Anthropic's research team found that vague delegation caused workers to duplicate each other's searches or miss important boundaries; its solution was to teach the orchestrator to specify objective, output format, source guidance, and scope boundaries for each subagent. If you do not define ownership, the agents will improvise it, and they are surprisingly willing to improvise badly. [93]

The second is **hallucinated ownership**: one agent assumes another has checked something when nobody has. Anthropic's long-running harness work counters this by forcing explicit feature-state tracking in JSON, end-to-end verification before marking status as passing, and progress updates in logs and commits. Claude Code's `TaskCompleted` hooks also exist precisely to stop tasks being marked complete without meeting actual criteria. [94]

The third is **silent intermediate failure**. Anthropic says debugging agents required full production tracing because otherwise they could not tell whether failure came from bad search queries, poor source choice, or tool issues. Managed Agents' top-level session stream is only a condensed view; to inspect what a called agent actually did, you have to look at the specific thread stream. OpenAI similarly recommends starting with traces when debugging agent behaviour. [95]

The fourth is **context bloat and memory pollution**. Anthropic's context-engineering post frames agent work as repeated curation of what enters the context window. Claude Code uses skills, subagents, `/compact`, and memory systems to keep sessions workable; Managed Agents advises many small memory files rather than a few large ones; Anthropic's long-horizon research systems summarise completed phases and externalise state before continuing. Shared memory can help, but stale or noisy shared memory can degrade the entire system. [96]

The fifth is **schema mismatch and brittle prompt contracts**. Anthropic's strict tool use exists because non-strict tool calls can produce incompatible types or missing fields; Microsoft's handoff executor filters internal handoff mechanics out of the forwarded history so those mechanics do not confuse downstream models; OpenAI likewise distinguishes when a handoff should own the reply versus when an agent should just act as a bounded tool. Reliable multi-agent systems nearly always move toward more explicit contracts over time. [97]

The sixth is **tool misuse and security failure**. Anthropic's trustworthy-agents post makes two points worth retaining: more open environments create more entry points for prompt injection, and more tools create more ways for an attacker to act once inside. Anthropic also stresses that a good model can still be undermined by a bad harness, a permissive tool, or an exposed environment. In the research-system post, Anthropic adds the more prosaic point that bad tool descriptions send agents down the wrong path. Both are forms of tool misuse, one malicious and one accidental. [98]

The seventh is **cost explosion and latency stacking**. Anthropic's docs explicitly warn that agent teams add coordination overhead and use significantly more tokens than a single session, even if parallelism can reduce wall-clock time. Anthropic's research system improved performance by parallelising both agents and tool calls, but that is not "free speed"; it is faster by spending more structured effort in parallel. [99]

The eighth is **evaluation difficulty**. Anthropic's recommendation is strong here: do not evaluate whether the agent followed exactly your intended process; evaluate whether it achieved the correct final state. OpenAI's eval guidance similarly says to start with traces while behaviour is still unstable. Multi-agent evaluation is hard because many different internal trajectories can still end in success. If your only test is "did the final prose sound plausible?", you are not evaluating the system; you are admiring it. [100]

Practical mitigations

Failure mode	Practical mitigation	Evidence
Coordination drift and duplicate work	Give each worker an explicit objective, expected output, source guidance, and task boundaries.	[93]
Agents mark work as done too early	Use explicit completion artifacts, task hooks, end-to-end verification, and durable state files.	[94]
Silent failures inside the	Instrument traces, inspect worker-level history, and	[95]

Failure mode	Practical mitigation	Evidence
team	store intermediate outputs.	
Context overflow and memory pollution	Summarise phases, isolate heavy work in subagents, prefer small focused memory files, use external artefacts.	[101]
Tool/schema brittleness	Use JSON Schema, strict tool mode, and narrower agent/tool contracts.	[102]
Prompt injection and over-permissioned agents	Limit permissions, keep tools scoped, preserve human approval points, and use safer environments.	[103]
Cost blow-up	Scale effort to task complexity; do not fan out by default; reserve teams for genuinely parallel work.	[104]
Maintenance brittleness	Version agents/configurations, use staged deployment, and treat agent systems as stateful infrastructure.	[105]

Blog post structure and open questions

Possible blog post table of contents

A strong long-form blog post on this topic should feel like a guided argument, not a bag of patterns.

One sensible narrative would be:

- **Why people reach for agent teams too early** — begin with the anti-hype premise that one agent plus good tools often beats a badly coordinated team. [106]
- **What an agent actually is** — use Anthropic's "model directs its own processes and tool use in a loop" framing, so the reader has the right baseline. [107]
- **The Anthropic stack for agentic systems** — walk from Messages API to Agent SDK to Managed Agents, then distinguish subagents, agent teams, and multiagent preview carefully. [108]
- **When one agent stops being enough** — explain the real triggers: context isolation, specialisation, parallel search, independent critique, human oversight. [109]
- **The four paradigms of agent teams** — describe orchestrator, pipeline, parallel specialists, and swarm as mental models, with honest caveats. [110]
- **Native Anthropic features versus building your own** — present the trade-offs as architecture, not ideology. [78]
- **How agent communication actually works** — establish the how/when/what framework, then explain MCP and A2A without conflating them. [92]
- **Why agent teams fail in production** — make this a full section, because it is the section most blog posts try very hard to avoid. [111]

- **A pragmatic decision framework** — finish by helping the reader decide between one agent, subagents, managed agents, and fully custom orchestration. [112]

Open questions and wording cautions

A few areas need careful wording in any published piece.

Anthropic's multi-agent story is still moving quickly. As of **27 April 2026**, Claude Code agent teams are experimental and Managed Agents multiagent is Research Preview. Those statuses should be date-stamped in the blog post so the piece does not age into accidental overclaiming. [113]

Claude Code teams and Managed Agents multiagent are not the same thing. One is a product feature for coordinating Claude Code sessions with shared local resources; the other is an API preview using coordinator and thread abstractions inside a hosted runtime. It would be misleading to collapse them into one undifferentiated "Claude multi-agent" capability. [114]

Anthropic's internal performance claims should be framed as internal evidence, not universal law. The claim that Anthropic's multi-agent research system beat a single-agent setup by **90.2%** is about Anthropic's internal research evaluation and a particular breadth-first task profile. It is useful evidence, but not proof that multi-agent is usually better. [115]

Portability needs nuance. The Agent SDK supports multiple provider back ends such as Bedrock, Vertex AI, and Azure Foundry, but Managed Agents is only available through the direct Claude API. So "Anthropic-native" does not always mean "portable Anthropic". [116]

There is not yet one universal inter-agent standard inside Anthropic's own product line. Claude Code teams use a mailbox/task-list model; Managed Agents multiagent uses threads and inter-thread events. That is not a criticism, merely a sign that the platform is still evolving. This point is an inference from the reviewed docs rather than a sentence Anthropic states directly. [117]

Confidence scores and other meta-signals are design choices, not protocol defaults. They can be helpful, but none of MCP, A2A, or the Anthropic/OpenAI tool specs reviewed here makes them a mandatory primitive. If you mention them, label them as a recommended pattern rather than a platform feature. [118]

---

[1] [4] [7] [38] [39] [40] [44] [45] [46] [47] [53] [54] [69] [106] [110] https://www.anthropic.com/research/building-effective-agents

https://www.anthropic.com/research/building-effective-agents

[2] [20] [22] [64] [74] [76] [77] [116] https://code.claude.com/docs/en/agent-sdk/overview

https://code.claude.com/docs/en/agent-sdk/overview

[3] [8] [26] [27] [31] [35] [43] [49] [52] [56] [57] [58] [79] [99] [112] [113] [114] [117] https://code.claude.com/docs/en/agent-teams

https://code.claude.com/docs/en/agent-teams

[5] [15] [90] [92] [118] https://modelcontextprotocol.io/specification/2025-03-26

https://modelcontextprotocol.io/specification/2025-03-26

[6] [11] [42] [48] [71] [81] [93] [95] [100] [101] [104] [111] [115] https://www.anthropic.com/engineering/multi-agent-research-system

https://www.anthropic.com/engineering/multi-agent-research-system

[9] https://code.claude.com/docs/en/sub-agents

https://code.claude.com/docs/en/sub-agents

[10] [32] https://code.claude.com/docs/en/memory

https://code.claude.com/docs/en/memory

[12] [21] [23] [33] [37] [62] [63] [70] [78] [108] https://platform.claude.com/docs/en/managed-agents/overview

https://platform.claude.com/docs/en/managed-agents/overview

[13] [98] [103] [107] https://www.anthropic.com/research/trustworthy-agents

https://www.anthropic.com/research/trustworthy-agents

[14] https://developers.openai.com/api/docs/guides/agents

https://developers.openai.com/api/docs/guides/agents

[16] https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/

https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/

[17] [61] [85] https://adk.dev/agents/multi-agents/

https://adk.dev/agents/multi-agents/

[18] https://learn.microsoft.com/en-us/agent-framework/workflows/orchestrations/

https://learn.microsoft.com/en-us/agent-framework/workflows/orchestrations/

[19] https://metr.org/blog/2026-03-25-red-teaming-anthropic-agent-monitoring/

https://metr.org/blog/2026-03-25-red-teaming-anthropic-agent-monitoring/

[24] [36] [66] [68] [75] [89] https://platform.claude.com/docs/en/managed-agents/multi-agent

https://platform.claude.com/docs/en/managed-agents/multi-agent

[25] [109] https://code.claude.com/docs/en/agent-sdk/subagents

https://code.claude.com/docs/en/agent-sdk/subagents

[28] https://code.claude.com/docs/en/features-overview

https://code.claude.com/docs/en/features-overview

[29] https://code.claude.com/docs/en/skills

https://code.claude.com/docs/en/skills

[30] https://code.claude.com/docs/en/hooks

https://code.claude.com/docs/en/hooks

[34] [72] [105] https://platform.claude.com/docs/en/managed-agents/migration

https://platform.claude.com/docs/en/managed-agents/migration

[41] https://learn.microsoft.com/en-us/agent-framework/overview/

https://learn.microsoft.com/en-us/agent-framework/overview/

[50] [51] [59] https://microsoft.github.io/autogen/stable/user-guide/core-user-guide/design-patterns/group-chat.html

https://microsoft.github.io/autogen/stable/user-guide/core-user-guide/design-patterns/group-chat.html

[55] [65] [88] [94] https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents

https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents

[60] https://a2a-protocol.org/latest/

https://a2a-protocol.org/latest/

[67] https://code.claude.com/docs/en/agent-sdk/observability

https://code.claude.com/docs/en/agent-sdk/observability

[73] https://platform.claude.com/docs/en/api/overview.md

https://platform.claude.com/docs/en/api/overview.md

[80] [102] https://platform.claude.com/docs/en/agents-and-tools/tool-use/define-tools

https://platform.claude.com/docs/en/agents-and-tools/tool-use/define-tools

[82] [83] https://developers.openai.com/api/docs/guides/agents/orchestration

https://developers.openai.com/api/docs/guides/agents/orchestration

[84] https://a2a-protocol.org/latest/topics/streaming-and-async/

https://a2a-protocol.org/latest/topics/streaming-and-async/

[86] https://learn.microsoft.com/en-us/agent-framework/workflows/orchestrations/handoff

https://learn.microsoft.com/en-us/agent-framework/workflows/orchestrations/handoff

[87] https://code.claude.com/docs/en/context-window

https://code.claude.com/docs/en/context-window

[91] https://a2a-protocol.org/latest/specification/

https://a2a-protocol.org/latest/specification/

[96] https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

[97] https://platform.claude.com/docs/en/agents-and-tools/tool-use/strict-tool-use

https://platform.claude.com/docs/en/agents-and-tools/tool-use/strict-tool-use